

First Loss: Evidence-Boundary Reasoning for Root-Cause Attribution

KuangYu Chen

Veridical Tech, Inc.

Xi'an University of Science and Technology Hi-tech College

rin@rin.red

Preprint revision v1.1.2, September 2026

Abstract

Failures in multi-stage software systems often become visible after the evidence needed for a correct result has already disappeared or changed meaning. Debugging from the final symptom can therefore blame a component that never received the relevant evidence. First Loss Methodology (FLM) treats an execution as a sequence of evidence transformations. *First Loss* marks the first boundary where evidence becomes unavailable to the next stage; *Invariant Loss* marks the first observed violation of identity, order, scope, time, provenance, authority, or lifecycle. Neither is automatically the root cause, which still requires intervention.

We formalize this distinction with segment-local definitions that cover acquisition boundaries, bounded intervals, nonlocal exceptions, and evidence reacquisition, and we state the attribution rule as falsifiable propositions. A controlled benchmark with 1,200 test cases and a 600-case replay window checks the internal behavior of the rules: removing the First Loss rule or the attribution boundary drops root-cause accuracy from 1.000 to 0.050. Three executable reference runtimes for retrieval-augmented generation, ETL, and build dependency resolution are evaluated under six stress profiles. Eighteen fault configurations replayed twenty times each yield 360 runs with 0.981 safe-attribution accuracy. Every exact-attribution error occurs under probe noise, where the method claims a wrong exact stage instead of a bounded interval; five-probe replay corrects seven such runs but reverses four correct ones, a net effect that is not statistically separable (exact McNemar $p = 0.549$). A 300-response GLM baseline reaches 0.920 overall attribution accuracy, yet only 12 of 60 cases receive five identical answers even at temperature zero; on the shared cases the model matches FLM only when at least one of its five repeats is accepted. Two production investigations illustrate rank loss and a compound acquisition–admission–assembly–enumeration failure. These results establish reproducibility and boundary behavior in controlled systems. They do not establish production-wide generality.

1 Introduction

A reader fails to cite a document. A build fails at the linker. A record is missing from a downstream table. The visible component is an obvious place to start looking, but it may be the wrong place to assign cause. In each example, the required evidence may have vanished earlier: a ranked document was reordered before a cap, an artifact version changed during resolution, or a join key was corrupted before the record reached the sink. Figure 1 shows the shape of such an incident: the observed failure sits at the reader, the evidence disappears at the cap, and the semantic corruption that caused the disappearance sits one stage earlier.

Fault localization usually ranks suspicious program locations from failing executions [7, 1]. Cause-transition and cause-effect work asks where a failing state first diverges from a passing one [15, 6]. Distributed tracing follows requests across service boundaries [13, 11], while provenance records where data came from and how it changed [2, 5]. FLM builds on these ideas but asks a narrower question: what may be blamed when a particular piece of evidence did not survive the path to the observed failure?

The method separates three locations that are easy to collapse into one:

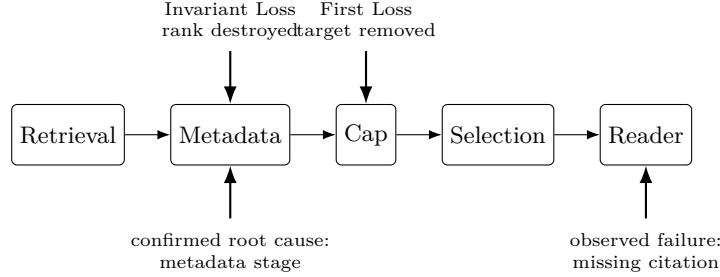


Figure 1: The motivating rank-loss incident. The failure is observed at the reader, but the evidence disappears at the cap and its ordering invariant is destroyed at the metadata stage. Downstream stages cannot be direct-path causes of losing evidence they never received; the confirmed root cause requires intervention at the metadata stage.

- **Invariant Loss:** the first observed semantic corruption;
- **First Loss:** the first boundary where the evidence becomes unavailable;
- **Root cause:** the factor whose controlled change removes the failure.

This separation gives FLM its main rule: a stage cannot be the direct-path cause of losing evidence that never reached it. The rule is intentionally narrower than “the downstream component is healthy.” Shared state, callbacks, concurrency, and hidden protocols can still create nonlocal causes.

This paper contributes (1) an evidence-lineage model with explicit loss and invariant boundaries, stated as falsifiable propositions; (2) rules for exact, bounded, nonlocal, and reacquired cases; (3) a confirmation protocol based on a frozen baseline and single-variable intervention; (4) two reproducible evaluations whose component-necessity ablations are reported in the main text, including three executable reference runtimes; and (5) two production investigations that expose different loss surfaces. The artifact is archived separately [3].

2 Related work

2.1 Fault localization, cause transitions, and algorithmic debugging

Spectrum-based fault localization uses coverage and test outcomes to rank suspicious program entities [7, 1]. It is useful when the target is a statement, branch, or method. FLM instead follows a named evidence item across system boundaries. The two approaches can be combined: FLM can first narrow the admissible evidence path, after which fault localization can rank code within the remaining stages.

Zeller’s cause-effect chains and Cleve and Zeller’s cause transitions identify differences that connect a failure to its origin [15, 6]. FLM shares the interest in early divergence but does not equate the first semantic divergence with the first disappearance. A record may keep flowing after its tenant, ordering, or version invariant has failed. Algorithmic debugging asks why a component returned a wrong value by comparing intended and actual behavior along the call hierarchy [12]; FLM asks a different question – why evidence that a component needed never arrived – and therefore attributes along data lineage rather than along the call tree. Delta debugging minimizes and isolates failure-inducing input differences [16]; its single-variable discipline is the ancestor of FLM’s minimal-probe rule, but FLM applies the discipline to pipeline configuration and evidence state rather than to program input.

2.2 Provenance, traces, and database causality

Data provenance distinguishes why an output exists from where its contributing values came from [2, 5]. FLM uses lineage as its identity backbone, then adds consumability and stage-specific invariants. Provenance alone does not say whether an output remained suitable for the next consumer.

Causal attribution inside databases computes which input tuples are responsible for an unexpected answer or non-answer [9]. That work formalizes counterfactual and actual responsibility at the tuple level within one query plan. FLM operates one level of abstraction higher, across ordered stage boundaries, and adds consumability, invariant surfaces, and an intervention protocol; the two are complementary, and database causality is a natural instrument once FLM has bounded the admissible stages.

Request-flow comparison and distributed tracing reconstruct cross-service execution paths [11, 13]. MicroRCA combines service dependencies, metrics, and anomaly information to localize performance faults [14]. FLM is not a replacement for those systems. It is an attribution discipline that can consume their traces when the incident depends on evidence survival.

2.3 LLM-based root cause analysis

Large language models are increasingly used to automate incident RCA. RCACopilot matches cloud incidents to handlers and predicts root-cause categories with an LLM narrative [4], and mABC coordinates multiple LLM agents with blockchain-inspired voting for microservice RCA [17]. These systems optimize for naming the cause; they do not separate the loss boundary from the cause or require intervention before a confirmed claim. FLM treats a trace-only model answer as a candidate generator, exactly the role of the real-model baseline in Section 4.3: useful for coverage, unsafe as an attribution rule.

2.4 Causal confirmation and RAG pipelines

Counterfactual reasoning distinguishes association from intervention [10]. FLM therefore labels a root cause *confirmed* only after a frozen fail-before/pass-after comparison and an independent replay. In retrieval-augmented generation, retrieved documents are explicit non-parametric evidence [8]; ranking, metadata enrichment, context caps, and assembly make that setting a natural example, but the method is not tied to RAG.

3 Method

3.1 Evidence transformations

Let a system instance be

$$\mathcal{M} = (\mathcal{E}, \mathcal{S}, \Omega), \quad (1)$$

where $\mathcal{E}$ is the evidence universe, $\mathcal{S} = (S_1, \dots, S_n)$ is an ordered path of transformations, and Ω is the observation model. An evidence item

$$E = (id, c, m, p, a, l) \quad (2)$$

contains identity, content, metadata, provenance, authority, and lifecycle state. A transformation may copy, split, merge, enrich, reorder, filter, or reject an item. Lineage identity is therefore a recoverable mapping, not object identity.

For a lineage representation of E after stage i , define

$$P_i(E) = \begin{cases} 1 & \text{if the representation is available and consumable downstream,} \\ 0 & \text{if it is unavailable or unusable,} \\ ? & \text{if } \Omega \text{ does not observe the boundary.} \end{cases} \quad (3)$$

Consumability matters. A byte string that survives storage but cannot be decoded by the next stage is absent for this analysis.

Definition 1 (Observed segment). *An observed segment of E is a maximal interval of stages $S_a, \dots, S_b$ such that Ω observes $P_a, \dots, P_b$ (the symbol ? occurs nowhere inside the interval) and either $a = 1$ or $P_{a-1} \neq ?$. A segment boundary is any transition between two segments or between a segment and an unobserved boundary.*

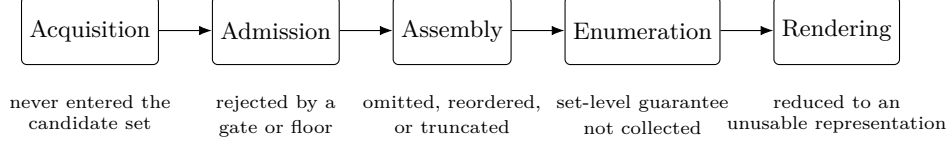


Figure 2: Operational loss surfaces. Each surface is a question about where the evidence predicate failed along the path; all of them map to the same First Loss and Invariant Loss definitions on a concrete segment.

Invariant checks require a specification source. For every invariant v checked at stage i , the expected value $I_{i,v}^{exp}(E)$ must come from a written contract – a schema, a policy file, or a documented stage guarantee – that is frozen before the trace is inspected. Expected values that are written after seeing the failure would make Invariant Loss unfalsifiable, so the confirmation protocol in Section 3.5 records the contract revision alongside the trace.

3.2 First Loss and Invariant Loss

Within one observed segment σ ,

$$FL_{\sigma}(E) = \min\{i \in \sigma : P_i(E) = 1 \wedge P_{i+1}(E) = 0\}. \quad (4)$$

The First Loss stage is S_{i+1} . Equation (4) has two edge cases that the method reports explicitly rather than hiding. If $P_a(E) = 0$ at the start of the segment, the item was already unavailable when the segment opens; FLM reports an *acquisition boundary* at S_a and does not claim a loss stage inside the segment. If no $1 \rightarrow 0$ transition occurs, no First Loss exists on σ and the item was continuously consumable.

If an item later returns through fallback retrieval or a side input, the return starts a new segment. Each segment is analyzed independently under Equation (4): the earlier loss is still reported for the failing path, the return is reported as reacquisition, and exclusion on the new segment never extends the previous segment’s boundary. Presence is not assumed monotone.

Let $I_{i,v}^{obs}(E)$ and $I_{i,v}^{exp}(E)$ be the observed and contracted values of invariant v after stage i . Then

$$IL(E) = \min\{i : \exists v, I_{i,v}^{obs}(E) \neq I_{i,v}^{exp}(E)\}. \quad (5)$$

An ordering violation can precede a cap that removes the item, so $IL(E) < FL(E)$. An expired item may remain present until the final output, in which case Invariant Loss exists without an earlier disappearance. Invariant Loss is a stage-level observation; it does not by itself name the predicate, configuration, or code defect that caused the violation.

3.3 Operational loss surfaces

The same definition applies at several operational surfaces. *Acquisition loss* means that required evidence never enters the candidate set. *Admission loss* means that a retrieved candidate is rejected by a gate. *Assembly loss* occurs when admitted evidence is omitted, reordered, or truncated while the final context is built. *Enumeration loss* means that relevant evidence reaches the consumer, but a question that requires a complete set is answered from an incomplete collection; the item is present, yet the set-level guarantee fails. *Rendering loss* means that the context reaches the answer stage but is reduced to a representation that cannot support the required response. These labels identify where to instrument the path; they do not replace the evidence predicate or turn a location into a confirmed cause. Figure 2 shows the surfaces along one path, and Table 1 maps them to the fault families used in the controlled benchmark and to the boundaries exposed by the two production investigations in Section 5.

A policy also needs an end-to-end predicate. A lane rule may reserve a candidate seat yet fail after another retrieval leg is merged. We therefore distinguish seat, filtering, ordering, and final-output guarantees. The relevant invariant is the one promised at the consumer boundary, not merely at an intermediate list.

Table 1: Loss surfaces mapped to synthetic fault families and production evidence.

Surface	Synthetic fault families	Production boundary
Acquisition	retrieval miss, budget miss	Case 2, boundary 1
Admission	filter mismatch, selector error, dead path	Case 2, boundary 2
Assembly	rank loss, positional truncation, duplicate identity	Case 1; Case 2, boundary 3
Enumeration	(no synthetic family)	Case 2, boundary 4
Rendering	temporal invalidity, metadata corruption	—

3.4 Attribution boundary and its limits

Suppose $\text{FL}(E) = k$ on an observed lineage segment and the failure propagates through that same evidence path. Stages after S_k are excluded as *direct-path causes* of losing E . This is the attribution boundary. It does not exclude four alternatives:

1. a later stage has an independent defect;
2. an upstream stage changes shared state used downstream;
3. an implicit callback or concurrent action bypasses the observed path;
4. the evidence is reacquired from another source.

When one of these mechanisms is plausible, FLM returns a cross-stage exception instead of a component-wide exclusion.

Definition 2 (Bounded attribution). *If $P_i(E) = 1$, $P_j(E) = 0$ for observed brackets $i < j$, and one or more boundaries between them are unobserved, the method reports the interval $(S_i, S_j]$ and nothing narrower. It must not select a hidden stage merely because that stage is expected from the architecture.*

The exclusion rule and its obligations are stated as propositions so that they can be falsified by the evaluation and by production practice:

Proposition 1 (Existence boundary). *If $P_i(E) = 0$, then E cannot directly appear as an input to S_{i+1} along the observed path. A downstream stage may still affect E through shared state, concurrency, or reacquisition, which is why the exclusion in Proposition 3 is limited to the direct path.*

Proposition 2 (Provenance recoverability). *If $P_i(E) = 1$ and Ω observes the segment, there must exist a finite, ordered lineage chain that maps the source identity of E to its representation at S_i , through copy, enrich, reorder, split, and merge. If no such chain is recoverable, the method reports a bounded result; it must not assert that the evidence arrived.*

Proposition 3 (Attribution bound). *If $\text{FL}(E) = k$ on the segment through which the failure propagates, then no stage $S_{k+1}, \dots, S_n$ may be reported as a confirmed root cause of losing E on that segment. The bound is falsified by any demonstrated nonlocal mechanism: shared state, a concurrent side effect, or an implicit protocol that lets a downstream stage destroy upstream evidence. In that case FLM downgrades its output to a cross-stage exception and narrows the exclusion to the direct path.*

Proposition 4 (Invariant priority). *If $\text{IL}(E) = j$ precedes $\text{FL}(E) = k$, the candidate set for the confirmed root cause must contain the predicate, configuration, or implementation that violates invariant v at S_j . A candidate set that contains only downstream stages violates the rule.*

3.5 Confirmation protocol

FLM distinguishes a candidate from a confirmed cause. Confirmation requires a reproducible failing baseline, a recorded First Loss or bounded interval, an explicit invariant or rejection predicate, a minimal counterexample, a single-variable intervention, fail-before/pass-after evidence, upstream checks, and an independent

Algorithm 1 FLM attribution on one observed lineage segment σ

Require: observed boundary states $\{P_i\}_{i \in \sigma}$; invariant checks $I_{i,v}^{obs}$ with frozen contract $I_{i,v}^{exp}$; recorded non-local markers

- 1: $IL \leftarrow \min\{i \in \sigma : \exists v, I_{i,v}^{obs} \neq I_{i,v}^{exp}\}$ $\triangleright$ or $\perp$
- 2: **if** $P_{\min \sigma} = 0$ **then**
- 3: **return** ACQUISITIONBOUNDARY($S_{\min \sigma}, IL$)
- 4: **end if**
- 5: **if** a nonlocal side effect is recorded at or before the loss transition **then**
- 6: **return** CROSSSTAGEEXCEPTION(IL)
- 7: **end if**
- 8: $FL \leftarrow \min\{i \in \sigma : P_i = 1 \wedge P_{i+1} = 0\}$ $\triangleright$ Eq. (4)
- 9: **if** FL exists **then**
- 10: **return** EXACT(FL, IL); exclude $S_{FL+1}, \dots$ on this segment
- 11: **else if** both observed brackets $i < j$ of $P_i = 1, P_j = 0$ exist around unobserved boundaries **then**
- 12: **return** BOUNDED(S_i, S_j) $\triangleright$ Definition 2
- 13: **else**
- 14: **return** NOLOSSONSEGMENT(IL)
- 15: **end if**

replay. Without intervention, the output remains a candidate set. Without replay, transient probe errors remain unresolved.

Algorithm 1 states the deterministic attribution step that precedes confirmation: it reads one observed segment and returns an exact boundary, an acquisition boundary, a bounded interval, or a cross-stage exception.

The confirmation level is also granular. An intervention that changes an entire metadata stage confirms a stage-level cause. Predicate-level confirmation needs the exact configuration, code revision, and before/after trace for the predicate itself.

4 Evaluation

The experiment protocol registered in the artifact [3] states four hypotheses: H1, First Loss localization beats component-level root cause analysis; H2, Invariant Loss separation beats existence-only provenance; H3, the False Downstream Attribution Rate drops under the attribution boundary; and H4, removing the First Loss rule, invariant checking, upstream exclusion, or minimal-probe discipline degrades attribution quality. This section maps each hypothesis to the evaluation that can decide it. H2 is decided by the controlled benchmark and its ablations; H3 is decided by the same benchmark, where only the boundary rule prevents downstream attribution; H4 is decided by the ablation tables of both benchmarks. H1, by contrast, is *not* decidable on the controlled benchmark, because the strongest baseline also receives complete traces and therefore localizes First Loss perfectly; H1 is decided by the executable runtimes, where traces are partial and noisy.

We ask four questions: whether FLM locates evidence loss, whether it separates semantic corruption from disappearance, whether it avoids unsafe downstream attribution, and whether intervention and replay change the strength of the result.

4.1 Controlled rule benchmark

The first benchmark freezes 1,200 test cases and 600 cases in a separate seed window. It covers 16 single-fault and four compound-fault families over one nine-stage abstract pipeline. Ground truth is generated from the injection specification before any method runs. This benchmark checks rule implementation and reproducibility, not domain generality, and its headline 1.000 scores should be read as an implementation check: any method that receives the complete trace and implements the same ground-truth vocabulary will match it. Appendix A reports the full five-method comparison, in which a deterministic trace-pattern baseline matches FLM exactly, as expected.

Table 2: Controlled-benchmark ablations, 1,200 test cases. Every ablated rule is necessary for at least one metric.

Variant	RCA stage	RCA confirmed	1-FDAR
FLM, full	1.000	1.000	1.000
– First Loss rule	.050	.050	.150
– invariant checking	.500	.500	1.000
– upstream exclusion	.050	.050	.150
– minimal probe	.432	.432	.644
– intervention	1.000	1.000	1.000
– independent replay	1.000	1.000	1.000
blind candidate selection	.987	.987	1.000

The informative result of this benchmark is the ablation in Table 2: every ablated rule is necessary for at least one metric. Removing the First Loss rule or the upstream exclusion collapses root-cause accuracy from 1.000 to 0.050, which is the quantified form of H4 and confirms H3: without the attribution boundary, the method blames downstream components exactly as often as the final-symptom baseline. Removing invariant checking drops Invariant Loss accuracy to 0.450, the level of an existence-only tracker (H2). Removing the minimal-probe discipline halves confirmed accuracy. The two confirmation ablations (no intervention, no replay) leave localization untouched – these rules change the *strength* of the claim, not its position, which is why the executable runtimes measure them separately.

The published run is exactly reproducible. We also corrected the bootstrap implementation in v1.1.0 so paired comparisons resample shared case indices rather than drawing each method independently. Point estimates are unchanged; confidence intervals are regenerated from the corrected procedure.

4.2 Executable heterogeneous runtimes

The second benchmark does not rename one common pipeline. It executes three separate implementations:

- a RAG path that ranks documents, enriches metadata, applies fallback retrieval, caps context, and answers;
- an ETL path that parses events, maps keys, replays dead letters, joins dimensions, and commits a window;
- a build path that resolves versions, reads a cache, falls back to vendored artifacts, links, and tests.

Each runtime ships one fault configuration per stress profile: local invariant-first failure, direct loss, hidden loss-stage observation, noisy probes, an explicit nonlocal side effect, and evidence reacquisition. Each configuration is replayed twenty times with distinct seeds, so the evaluation totals 18 configurations $\times$ 20 repetitions = 360 runs. The repetitions measure attribution robustness under re-execution – probe noise and instrumentation – rather than sampling of independent faults, and the statistics below therefore report both the run level and the configuration level.

Safe attribution requires an exact answer for fully observed local paths, a containing interval for hidden observations, an explicit exception for nonlocal effects, and segment-aware handling of reacquisition. Exact FLA, RCA, and confirmation are defined only on the 180 eligible-exact runs (local, direct, and noisy profiles); the bounded, exception, and reacquired profiles are scored by safe attribution and overclaim alone. Table 3 reports the aggregate and Table 4 the per-profile breakdown.

Three observations follow from the profile structure. First, the bounded, exception, and reacquisition rules behave as designed: FLM never claims an exact stage on a hidden, nonlocal, or reacquired path, so its safe-attribution failures cannot come from those profiles. Second, intervention changes only claim strength, not localization: FLM with and without intervention disagree on zero of 360 runs (exact McNemar $p = 1$), while confirmed accuracy falls from .961 to .000. Third, all exact-attribution errors concentrate in the noisy profile, which the ablation comparison in Table 5 quantifies.

Table 3: Executable-runtime evaluation, 360 runs (18 fault configurations $\times$ 20 replay repetitions). Exact metrics are over the 180 eligible-exact runs.

Method	Safe	FLA	RCA	Confirmed	Overclaim
Final symptom	.000	.000	.000	.000	1.000
Component heuristic	.000	.944	.000	.000	1.000
FLM, no intervention	.981	.961	.961	.000	.000
FLM, no replay	.972	.944	.944	.944	.000
FLM	.981	.961	.961	.961	.000

Table 4: Safe attribution by stress profile (three runtimes pooled; 60 runs per profile). Exact FLA is shown where the profile is eligible-exact.

Profile	Final symptom	Component	FLM –interv.	FLM –replay	FLM
Local	.000	.000	1.000	1.000	1.000
Direct	.000	.000	1.000	1.000	1.000
Partial	.000	.000	1.000	1.000	1.000
Noisy	.000	.000	.883	.833	.883
Nonlocal	.000	.000	1.000	1.000	1.000
Reacquired	.000	.000	1.000	1.000	1.000
Exact FLA (eligible only)					
Local / Direct	.000	1.000	1.000	1.000	1.000
Noisy	.000	.833	.883	.833	.883

Error taxonomy. Every safe-attribution failure of the full method – 7 of 360 runs – is a noisy-probe run in which FLM claims a wrong *exact* stage: the RAG probes misled it to place First Loss at *answer* instead of *cap* (3 runs), the ETL probes at *sink* instead of *join* (2 runs), and the build probes at *test* instead of *link* (2 runs). In each case the honest output was a bounded interval or an uncertain status, so these are overconfident-exact errors, not wrong-in-principle answers. The replay ablation shows the deeper pattern. Five-probe replay corrects 7 such runs, but it also reverses 4 runs that the single-pass method answered correctly: majority aggregation over noisy probes can itself manufacture an overconfident exact claim. Replay is therefore not monotone under probe noise, and the net benefit (0.981 vs 0.972) is not statistically separable at 18 configurations (Table 5). The protocol-level conclusion is that probe-noise disciplines must include an exactness guard – aggregate agreement below a threshold must downgrade the answer to bounded – which the current reference implementation does not yet enforce.

The component heuristic locates First Loss on fully observed paths (FLA .944) yet has zero safe attribution and overclaim 1.000, because it freely blames stages downstream of the loss boundary. This is the executable-runtime form of H1 and H3: localization without the boundary rule is unsafe, and the boundary rule, not the trace, supplies the safety.

4.3 Real-LLM baseline

The real-model baseline draws 60 cases from the 360-run pool by round-robin stratification over the 18 runtime/profile strata (six strata contribute 4 cases, twelve contribute 3) and repeats each case five times. It hides profile and ground truth, fixes the prompt and output contract, and records the requested and provider-reported model identifiers, temperature, prompt hash, response identifier, usage, latency, raw output, and per-run scores. The model does not receive the FLM boundary rule.

We requested `glm-5.3-flash` through an OpenAI-compatible Responses endpoint; the provider reported `glm-5-3-flash`. The gateway accepted but did not enforce strict JSON Schema, so two compatibility pilots (a six-case smoke test) were excluded. We then froze prompt version `llm-rca-v1.1.1`, used JSON-object mode with exact field validation, temperature 0.0, and a 300-token output budget, and ran 300 requests. The final run consumed 218,555 input and 90,129 output tokens. Metrics are defined exactly as in the

Table 5: Paired comparisons on safe attribution. McNemar p treats the 360 runs as paired binary outcomes; the clustered bootstrap resamples the 18 fault configurations and reflects the dependence of repetitions.

Pair	Corrected	Reversed	Exact McNemar p (clustered 95% CI of Δ)
FLM vs FLM –replay	7	4	.549 ([−.006, +.028])
FLM vs FLM –intervention	0	0	1 ([.000, .000])

Table 6: Real-model baseline: 60 cases with five runs each (300 runs), Wilson 95% intervals.

Metric	n	Result [95% CI]
Attribution accuracy	300	.920 [.884, .946]
Exact-path FLA / RCA	150	.987 [.953, .996] / .973 [.933, .990]
Exact-path ILA	150	.813 [.743, .868]
Status accuracy	300	.927 [.891, .951]
Risky-case overclaim rate	150	.047 [.023, .093]
Five-run exact agreement	60	.200 [.118, .318]
Final schema adherence	300	1.000 [.987, 1.000]

runtime benchmark: attribution accuracy over all 300 runs; exact-path FLA, ILA, and RCA over the 150 eligible-exact runs; overclaim over the 150 risky runs. Table 6 adds Wilson 95% intervals.

Three findings deserve emphasis. First, Invariant Loss is the model’s weak boundary: all 28 IL errors among the 150 eligible runs are hallucinations on direct-loss cases where no invariant is violated – the model reports an IL stage in 19 of these runs (for example, placing the invariant loss at *link* or *join*, the same stage where evidence disappears). The model conflates disappearance with semantic corruption, which is precisely the distinction FLM’s two definitions separate. Second, exact-path RCA (.973 [.933, .990]) is statistically indistinguishable from FLM’s .961 [.922, .981] on the eligible runs; the apparent difference is within noise and the sample is small. Third, at temperature 0.0 only 12 of 60 cases receive five identical answers (per-case distribution: 12 cases with one distinct answer, 25 with two, 18 with three, 5 with four). Provider-side nondeterminism, not sampling temperature, dominates the variance. The majority-vote root-cause accuracy over the five repeats is .817, but because this analysis was not preregistered we report it as exploratory rather than as a headline metric; a preregistered self-consistency protocol is required before voting can serve as an attribution rule.

Comparison with FLM. On the 60 shared cases the two methods are close but differ in kind: FLM answers 59 of 60 correctly (safe attribution), the model answers 45 of 60 when all five repeats must agree and 60 of 60 when a single correct repeat suffices. The union is complete – every case is solved by FLM or by at least one model repeat – but nothing in the model’s output identifies *which* repeat is correct without ground truth, whereas FLM’s answer is deterministic and its single failure is exactly the noisy-probe overconfidence described in Section 4.2. The comparison is also asymmetric in confirmation semantics: the model’s answer requires no intervention, so its RCA numbers are candidate-level claims. This is not a model leaderboard; it is evidence that a trace-only LLM is a strong candidate generator but needs an attribution rule and a confirmation protocol to become safe.

5 Production investigations

5.1 Rank loss before a cap

The motivating incident occurred in the SPM-Polaris recall pipeline. Vector retrieval returned ranked evidence. Metadata enrichment preserved the records but not their order. A downstream cap then removed the target by position. The target never reached selection, gating, assembly, or the reader.

The observed Invariant Loss was the metadata stage, and First Loss occurred at the cap. Preserving rank across metadata lookup, with the query, corpus, retrieval parameters, and cap fixed, removed the

failure in another process and time window. The released replay supports a confirmed *stage-level* cause at metadata enrichment. Because the public artifact omits the production query, corpus revision, code revision, and before/after payloads, the specific predicate remains a candidate rather than a publicly reproducible confirmed cause.

5.2 Compound loss in a memory-recall path

A later field investigation began with a fast recall mode that returned short field-like atoms while a deeper mode materialized source spans. Repeated queries initially suggested one defect: recently captured debugging text, including the query and a wrong answer, could outrank committed memory. The investigation did not stop there. Frozen-query replays showed that the same request could fail at three different boundaries as the candidate pool changed.

First, the committed item could be crowded out before selection, an acquisition loss. Second, an intact atom could enter the candidate set and then fail a semantic-support gate, an admission loss. Third, a span retrieval leg was merged after lane ordering and could therefore bypass the promised precedence, an assembly-level contract loss. Separate enumeration questions exposed a fourth boundary: relevant evidence reached the reader, but the selector did not collect a complete set of scattered mentions – an enumeration loss in the sense of Section 3.3. Treating all four outcomes as “recall failure” would have assigned one symptom to unrelated mechanisms.

The causal checks were staged. New diagnostics first exposed gate decisions and failure kinds. A paired lane-policy probe then held the tenant, query, and two candidate sources fixed while changing only the policy. Applying the policy after candidate and span evidence were merged restored the final ordering guarantee. A separate intervention changed semantic admission, and capture governance plus cross-source deduplication removed the query echo without suppressing committed evidence. The fast path still retained a documented quality boundary; not every weak result was reclassified as a bug.

This case also shows why replay must freeze more than code. Candidate-pool membership, source timestamps, capture state, retrieval depth, and evidence limits are part of the causal context. Without them, a changed result across two runs is evidence of drift, not evidence that the answer stage is nondeterministic. The public description is deliberately redacted and supports stage-level attribution only.

6 Threats to validity

Construct validity. The first benchmark encodes FLM-compatible ground truth, so its perfect result is an implementation check; the ablations, not the headline scores, carry its evidential weight. The executable runtimes reduce that coupling but remain authored reference systems. Safe attribution is a method-specific outcome and should not replace end-to-end incident resolution time.

Internal validity. Faults and probes are controlled, and the same authors designed the method and evaluation. The 360 runtime runs share one fault configuration per runtime/profile, so repetitions are dependent within a configuration; we report clustered intervals alongside run-level tests and avoid configuration-level significance claims. The final real-LLM prompt was frozen after two compatibility pilots, not before any contact with the endpoint. Those pilots are excluded and documented, but prompt adaptation can bias the result. The temperature-zero agreement failure shows that the serving stack contributes nondeterminism that the experiment does not control, and the gateway’s provider-reported model ID is not a versioned model snapshot. A preregistered external evaluation would reduce all three risks.

External validity. Three executable runtimes are more informative than seven renamed instances, but 18 fault configurations is a small population, the runtimes are deterministic apart from injected probe noise, and the study does not cover production concurrency, organizational handoffs, or unknown ground truth. Two production investigations show plausibility, not prevalence. The second is redacted, was conducted by the system’s developers, and is not independently reproducible from the public artifact.

Conclusion validity. Confidence intervals and McNemar tests describe repeated benchmark cases, not a random sample of all incidents. At 18 fault configurations, the replay ablation’s net effect cannot be statistically separated from zero; we report the corrected/reversed counts so readers can see the mechanism instead of a bare p-value. Multiple baseline comparisons should be treated as secondary. Very small exact *p*-values are reported as bounds rather than numerical zero.

7 Discussion

FLM is most useful when evidence has a stable or recoverable identity, stages are ordered along a relevant path, invariants can be checked, and at least a few boundaries can be observed or replayed. When those conditions fail, the honest output is an interval or an exception. That is not a failed diagnosis. It is the boundary of what the trace supports.

The method also changes the order of work. Engineers first ask whether the evidence reached a stage, then whether it still meant the same thing, and only then whether that stage behaved correctly. Policies are checked at the final consumer boundary after all retrieval legs are merged. This ordering does not make causal reasoning automatic, but it stops a common mistake: investigating a component as the cause of losing input it never received.

8 Conclusion

First Loss is an attribution boundary, not a root cause. Invariant Loss can occur earlier, and intervention supplies the causal claim that traces alone cannot. By limiting exclusion to an observed lineage segment, reporting bounded intervals, and recognizing nonlocal and reacquired paths, FLM turns a debugging intuition into a rule that can fail safely. The current evidence supports reproducibility and controlled boundary behavior. Production generality remains an empirical question.

Artifact and disclosures

The preprint and reproducibility artifact are archived in the Zenodo version chain at doi:10.5281/zenodo.22704462. The archive contains the frozen prompts and raw responses, the per-run score files, and a post-hoc analysis script (`analyze_results.py` in the benchmark directory) that regenerates every number, interval, and test reported in Section 4 from the archived result files into per-benchmark `ANALYSIS.md` reports. Code is licensed under MIT. Benchmark data and paper text are licensed under CC BY 4.0. The author designed the method, validated the experiments, and approved the manuscript. AI assistants supported prose drafting, code implementation, and experiment organization. No human-subject experiment was conducted. No external funding is reported.

A Controlled benchmark: full method comparison

Table 7 reports the five-method comparison on the controlled benchmark. It is retained for completeness and reproducibility; the ablations in Table 2 carry the evidential weight of this benchmark.

Table 7: Controlled benchmark, 1,200 test cases: full method comparison.

Method	FLA	ILA	RCA	Confirmed	1-FDAR
Final symptom	.150	.450	.050	.050	.150
Component RCA	1.000	1.000	.420	.420	.578
Existence only	1.000	.450	.500	.500	1.000
Trace pattern	1.000	1.000	1.000	1.000	1.000
FLM	1.000	1.000	1.000	1.000	1.000

References

- [1] Rui Abreu, Peter Zoetewij, Rob Golsteijn, and Arjan J. C. van Gemund. A practical evaluation of spectrum-based fault localization. *Journal of Systems and Software*, 82(11):1780–1792, 2009.

- [2] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, pages 316–330. Springer, 2001.
- [3] KuangYu Chen. First loss methodology: Benchmark and reproducibility artifact, 2026.
- [4] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. Automatic root cause analysis via large language models for cloud incidents, 2023. RCACopilot.
- [5] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.
- [6] Holger Cleve and Andreas Zeller. Locating causes of program failures. In *Proceedings of the 27th International Conference on Software Engineering*, pages 342–351, 2005.
- [7] James A. Jones, Mary Jean Harrold, and John Stasko. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering*, pages 467–477, 2002.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [9] Alexandra Meliou, Wolfgang Gatterbauer, Katherine F. Moore, and Dan Suciu. The complexity of causality and responsibility for query answers and non-answers. *Proceedings of the VLDB Endowment*, 4(1):34–45, 2010.
- [10] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition, 2009.
- [11] Raja R. Sambasivan, Alice X. Zheng, Michael De Rosa, Elie Krevat, Spencer Whitman, Michael Stroucken, William Wang, Lianghong Xu, and Gregory R. Ganger. Diagnosing performance changes by comparing request flows. In *8th USENIX Symposium on Networked Systems Design and Implementation*, pages 43–56. USENIX Association, 2011.
- [12] Ehud Y. Shapiro. *Algorithmic Program Debugging*. MIT Press, 1983.
- [13] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, 2010.
- [14] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. MicroRCA: Root cause localization of performance issues in microservices. In *2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9, 2020.
- [15] Andreas Zeller. Isolating cause-effect chains from computer programs. In *Proceedings of the 10th ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 1–10, 2002.
- [16] Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.
- [17] Wei Zhang, Hongcheng Guo, Jian Yang, Zhoujin Tian, Yi Zhang, Chaoran Yan, Zhoujun Li, Tongliang Li, Xu Shi, Liangfan Zheng, and Bo Zhang. mABC: Multi-agent blockchain-inspired collaboration for root cause analysis in micro-services architecture. *arXiv preprint arXiv:2404.12135*, 2024.